

SMART TIMETABLE GENERATOR PORTAL

A Rule-Based Web Application for Institutional Timetable Management

Team Members

- Shaziya Fathima S F – Team Lead
 - Harini V – Backend Developer
 - Manjura M – Frontend Developer
 - Keerthanasri S – Timetable Logic Developer
-

ABSTRACT

The Smart Timetable Generator Portal is a web-based application developed to simplify and digitize timetable management in educational institutions. The system provides role-based access for Administrators, Faculty, and Students, enabling efficient management of departments, faculty, subjects, classrooms, time slots, and timetables.

Built using the MERN stack (MongoDB, Express.js, React, and Node.js), the application ensures secure authentication using JWT and bcrypt. The scheduling process is entirely rule-based, supporting workflows such as guest lecture management, substitute faculty allocation, and timetable filtering. By centralizing scheduling operations, the system reduces manual effort, minimizes conflicts, and improves overall administrative efficiency.

1. INTRODUCTION

Timetable preparation is a critical administrative task that involves allocating faculty, classrooms, subjects, and time slots without conflicts. Traditional methods using spreadsheets are time-consuming, difficult to maintain, and prone to errors.

The Smart Timetable Generator Portal provides a centralized platform to manage institutional scheduling efficiently. It offers secure role-based access, simplifies timetable management, and stores all scheduling data in a centralized MongoDB database, making updates faster and more reliable.

2. PROBLEM STATEMENT

Educational institutions often rely on manual methods for timetable preparation, leading to scheduling conflicts, inefficient resource allocation, and increased administrative workload.

The objective of this project is to develop a centralized, rule-based Smart Timetable Generator Portal that automates timetable management while providing secure role-based access for Administrators, Faculty, and Students.

3. OBJECTIVES

- Digitize timetable management.
 - Reduce scheduling conflicts.
 - Provide secure role-based authentication.
 - Centralize institutional data.
 - Simplify faculty and classroom allocation.
 - Support guest lectures and substitute faculty workflows.
 - Enable timetable filtering and reporting.
-

4. EXISTING SYSTEM

The existing timetable preparation process is mostly manual and spreadsheet-based. It requires significant administrative effort and often results in scheduling conflicts, duplicate resource allocation, and delayed updates.

Limitations

- Manual timetable preparation
- High possibility of errors

- No centralized database
 - Difficult timetable modifications
 - Limited access control
 - Time-consuming report generation
-

5. PROPOSED SYSTEM

The Smart Timetable Generator Portal is a centralized web application that allows administrators to manage institutional scheduling through a secure interface. Faculty and students can view relevant timetables based on their roles.

The system manages departments, faculty, subjects, classrooms, time slots, and timetable entries while supporting guest lectures, substitute faculty allocation, and timetable filtering using predefined scheduling rules.

6. TECHNOLOGY STACK

Technology	Purpose
React + Vite	Frontend Development
Tailwind CSS	UI Design
Node.js	Backend Runtime
Express.js	REST APIs
MongoDB Atlas	Database
Mongoose	ODM
JWT	Authentication
bcryptjs	Password Encryption
Git & GitHub	Version Control
Thunder Client	API Testing

7. SYSTEM ARCHITECTURE

The application follows a three-tier architecture consisting of the frontend, backend, and database. The React frontend communicates with the Express backend through REST APIs. Business logic is handled using controllers and middleware, while MongoDB Atlas stores all application data through Mongoose models.

Figure 7.1: System Architecture



8. MODULE DESCRIPTION

The Smart Timetable Generator Portal is divided into independent modules, each responsible for a specific functionality. This modular design improves maintainability, scalability, and ease of development.

8.1 Authentication Module

This module provides secure user authentication and authorization. It allows users to register, log in, and access features based on their assigned roles (Administrator, Faculty, or Student). Passwords are securely encrypted using bcrypt, while JWT is used for session authentication.

Features

- User Registration
 - User Login
 - JWT Authentication
 - Role-Based Access Control
-

8.2 Department Management

This module enables administrators to manage academic departments by adding, updating, viewing, and deleting department records.

Features

- Add Department
 - Update Department
 - Delete Department
 - View Departments
-

8.3 Faculty Management

The Faculty Management module stores faculty information including department, designation, contact details, and subject allocation.

Features

- Add Faculty
 - Update Faculty
 - Delete Faculty
 - View Faculty Details
-

8.4 Subject Management

This module manages subject information such as subject code, credits, department, and semester.

Features

- Add Subject
 - Edit Subject
 - Delete Subject
 - View Subjects
-

8.5 Classroom Management

This module maintains classroom information including room number, seating capacity, and classroom type.

Features

- Add Classroom
 - Edit Classroom
 - Delete Classroom
 - View Classroom Details
-

8.6 Time Slot Management

This module manages the available teaching time slots used during timetable generation.

Features

- Create Time Slots
 - Update Time Slots
 - Delete Time Slots
 - View Available Time Slots
-

8.7 Timetable Management

This is the core module of the application. It allows administrators to create, update, search, and manage timetable entries using predefined scheduling rules.

Features

- Generate Timetable
 - View Timetable
 - Edit Timetable
 - Search & Filter Timetable
-

8.8 Guest Lecture Management

This module allows administrators to schedule guest lectures by allocating a free time slot or replacing an existing class. Cancelled classes are stored as pending makeup classes.

Features

- Schedule Guest Lecture
 - Allocate Free Slot
 - Reschedule Existing Class
 - Track Pending Makeup Classes
-

8.9 Substitute Faculty Management

When a faculty member is unavailable, administrators can assign an available substitute faculty member and automatically update the timetable.

Features

- Faculty Leave Processing
 - Substitute Allocation
 - Timetable Update
 - Student Notification
-

8.10 Reports & Notifications

The system supports report generation and notification management for institutional scheduling activities.

Reports

- Faculty Workload
- Department Timetable
- Room Utilization
- Subject Allocation
- Complete Timetable

Notifications

- Timetable Updates
 - Guest Lecture Notifications
 - Substitute Faculty Notifications
 - Schedule Change Alerts
-

9. DATABASE DESIGN

The Smart Timetable Generator Portal uses **MongoDB Atlas** as its cloud database. The database is designed using separate collections for each major entity, with relationships maintained through **MongoDB ObjectId references**.

The database structure minimizes data redundancy while ensuring efficient retrieval of timetable-related information.

Figure 9.1: ER Diagram

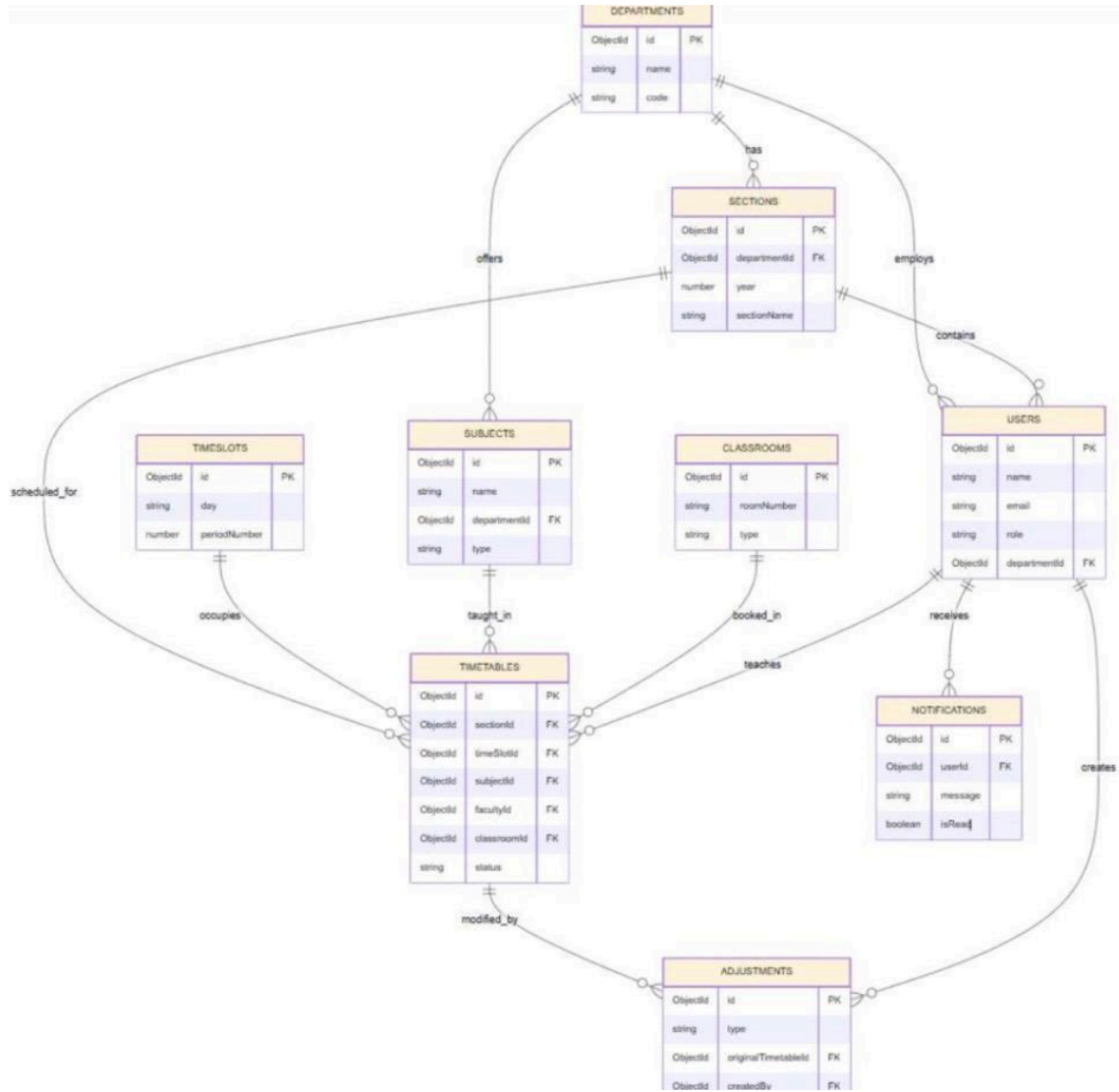
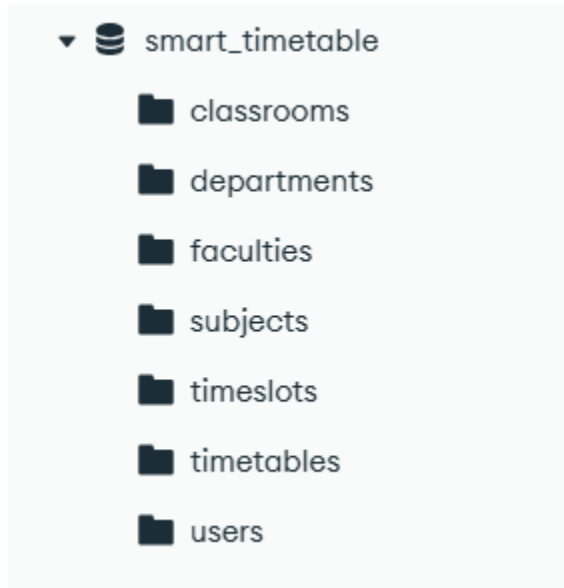


Figure 9.2: MongoDB Collections



9.1 Database Collections

Collection	Description
Users	Stores user authentication details
Departments	Department information
Faculty	Faculty details
Subjects	Subject information
Classrooms	Classroom details
TimeSlots	Teaching time slots
Timetables	Timetable entries

Collection Relationships

- One Department can have multiple Faculty members.

- One Department can offer multiple Subjects.
- One Faculty can handle multiple Subjects.
- One Classroom can be allocated to multiple timetable entries at different time slots.
- One Time Slot can be assigned to multiple classes on different days.
- Each Timetable entry references a Department, Faculty, Subject, Classroom, and Time Slot.

10. API DOCUMENTATION

The backend exposes REST APIs for managing application resources and communication between the frontend and the database.

Method	Endpoint	Purpose
POST	<code>/api/auth/register</code>	Register User
POST	<code>/api/auth/login</code>	User Login
GET	<code>/api/departments</code>	Retrieve Departments
POST	<code>/api/departments</code>	Add Department
GET	<code>/api/faculty</code>	Retrieve Faculty
POST	<code>/api/faculty</code>	Add Faculty
GET	<code>/api/subjects</code>	Retrieve Subjects
POST	<code>/api/subjects</code>	Add Subject
GET	<code>/api/classrooms</code>	Retrieve Classrooms
POST	<code>/api/classrooms</code>	Add Classroom
GET	<code>/api/timetables</code>	Retrieve Timetable
POST	<code>/api/timetables</code>	Generate Timetable

Figure 10.1: Thunder Client – Register API


```
User submits email/password
    ↓
Server verifies password against bcrypt hash
    ↓
JWT issued (contains user id, role)
    ↓
Token stored client-side, sent on every future request
    ↓
Redirected to role-specific dashboard (Admin/Faculty/Student)
```

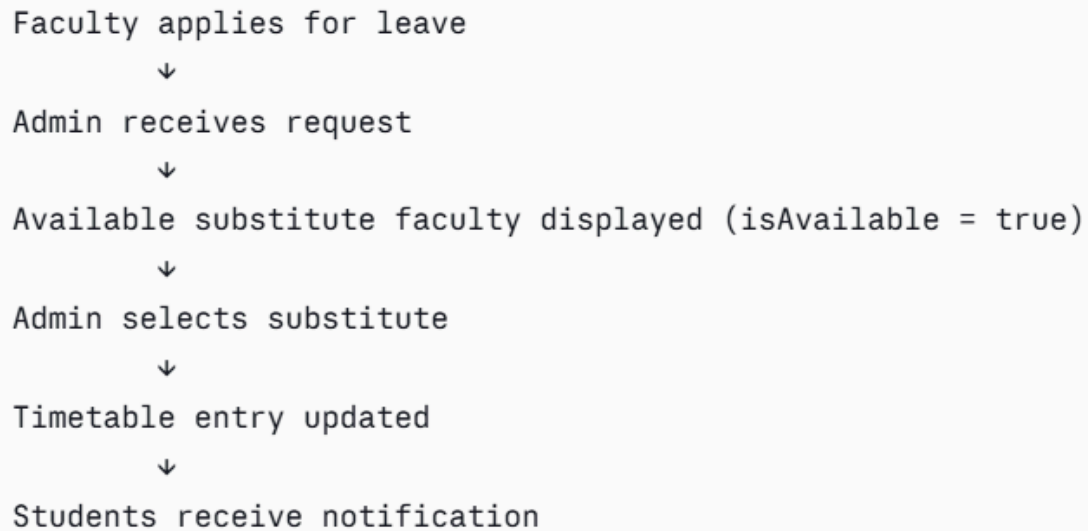
11.2 Timetable Generation Workflow

The administrator creates departments, faculty, subjects, classrooms, and time slots before generating the timetable. The system validates scheduling rules, checks for conflicts, and stores the generated timetable in the database.

11.3 Faculty Leave & Substitute Workflow

When a faculty member applies for leave, the administrator reviews the request and assigns an available substitute faculty member. The timetable is updated automatically, and students are notified of the changes.

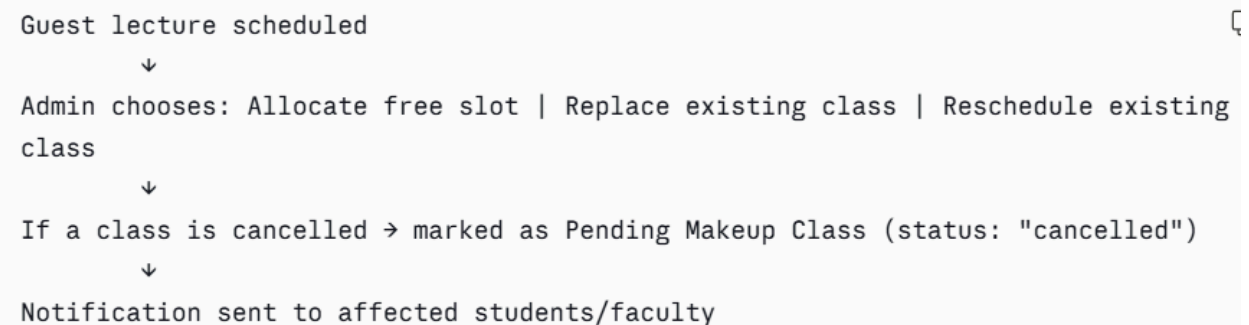
Figure 11.3: Substitute Faculty Workflow



11.4 Guest Lecture Workflow

The administrator schedules a guest lecture by assigning a free time slot or replacing an existing class. If a scheduled class is replaced, it is recorded as a pending makeup class for future rescheduling.

Figure 11.4: Guest Lecture Workflow



12. TESTING

The application was tested to ensure that all modules function correctly and that data is securely stored and retrieved from the database.

Test Case	Expected Result	Status
User Registration	User created successfully	✓ Passed
User Login	JWT token generated	✓ Passed
Add Department	Department stored in database	✓ Passed
Add Faculty	Faculty record created	✓ Passed
Add Subject	Subject added successfully	✓ Passed
Add Classroom	Classroom stored successfully	✓ Passed
Add Time Slot	Time slot created	✓ Passed
Generate Timetable	Timetable entry created	✓ Passed
Role-Based Access	Authorized users only	✓ Passed

13. CHALLENGES FACED

During the development of the Smart Timetable Generator Portal, the team encountered several technical and collaborative challenges.

Challenges

- Learning the MERN stack while developing the project.
- Setting up MongoDB Atlas and establishing a secure database connection.
- Resolving MongoDB SRV DNS connection issues.
- Understanding Git and GitHub collaboration workflows.
- Managing backend and frontend integration.
- Designing a scalable database schema.

- Coordinating development among multiple team members within a limited hackathon duration.

Solutions

- Followed modular development practices.
 - Used Git branches for independent feature development.
 - Established a clear project structure and defined responsibilities for each team member.
 - Tested APIs using Thunder Client before frontend integration.
 - Used MongoDB Atlas with Mongoose for reliable data management.
-

14. FUTURE ENHANCEMENTS

The current system provides a strong foundation for institutional timetable management. Future enhancements may include:

- Automated timetable optimization using advanced scheduling rules.
 - Email and SMS notifications for timetable updates.
 - Attendance management integration.
 - Calendar synchronization.
 - Multi-campus support.
 - Export timetables as PDF or Excel.
 - Analytics dashboards for workload and resource utilization.
-

15. CONCLUSION

The Smart Timetable Generator Portal successfully digitizes the timetable management process by providing a centralized, secure, and user-friendly web application. The system simplifies scheduling, reduces manual effort, minimizes conflicts, and improves coordination between administrators, faculty, and students.

Developed using the MERN stack, the application demonstrates the effective use of modern web technologies, secure authentication, RESTful APIs, and cloud-based database management. The modular architecture ensures scalability and maintainability, making the system suitable for future enhancements and real-world institutional use.

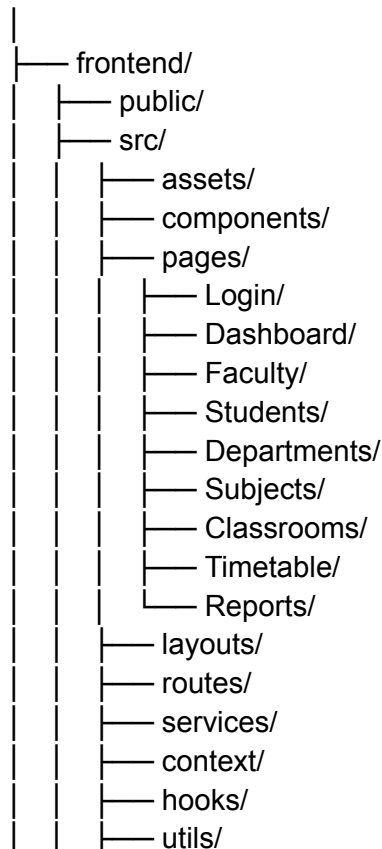
REFERENCES

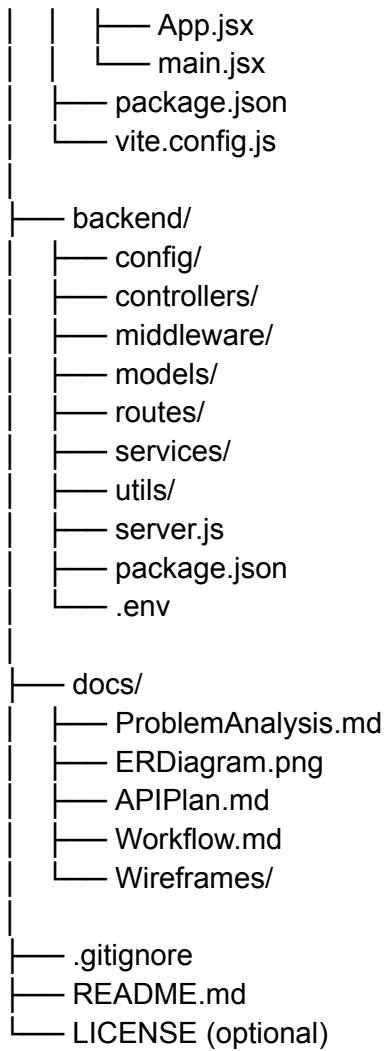
1. React Documentation – <https://react.dev/>
 2. Vite Documentation – <https://vite.dev/>
 3. Tailwind CSS Documentation – <https://tailwindcss.com/docs>
 4. Node.js Documentation – <https://nodejs.org/docs/latest/api/>
 5. Express.js Documentation – <https://expressjs.com/>
 6. MongoDB Atlas Documentation – <https://www.mongodb.com/docs/atlas/>
 7. Mongoose Documentation – <https://mongoosejs.com/docs/>
 8. JSON Web Token Documentation – <https://jwt.io/>
 9. Git Documentation – <https://git-scm.com/doc>
 10. GitHub Documentation – <https://docs.github.com/>
-

APPENDIX

Project Structure

SmartTimetablePortal/





GitHub Repository

Repository URL: <https://github.com/Shaziya2007/smart-timetable-ai>
